

COL1000: Introduction to Programming

Lecture 11-12

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

So far in the course

**Programs using int/float/bool/string variables,
if-else statements, for/while loops**

**Church-Turing Thesis : whatever computational problems can
be solved (in any programming language), can be solved
using whatever we have learnt in the course so far.**

So what's left for the rest of the semester?

Focus for next few lectures:
Efficiency of code, and better organisation of code

A recap of three programs seen in the course

Primality testing

A positive number n is prime if the only numbers that divide it are 1 and n . Write a program that takes as input n , and outputs “Yes” if n is prime.

```
n = int(input("enter number: "))  
  
i = 2  
  
prime = True  
  
while(i < n):  
    if (n%i == 0):  
        prime=False  
        break  
  
    i = i+1  
  
if(prime == True):  
    print(n,"is prime")  
else:  
    print(n,"is not prime")
```

Output

```
enter number: 15  
15 is not a prime
```

Output

```
enter number: 41  
41 is a prime
```

Greatest common divisor of two numbers

Write a program that takes as input two positive integers a , b and prints the largest g such that g divides both a and b .

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))

g = 1
gcd = 1

while ((g <= a) and (g <= b)):
    # the gcd has to be a number between 1 and min(a,b)
    # we will search this space sequentially,
    # starting with 1

    if ((a%g == 0) and (b%g == 0)):
        # if g divides both
        gcd = g

    g = g+1

print("GCD is",gcd)
```

Output

```
enter number: 15
enter number: 33
GCD is 3
```

Output

```
enter number: 49
enter number: 21
GCD is 7
```

Square root of a number

Write a program that takes as input a positive integer n , and prints the smallest y such that $y * y > n$

```
n = int(input("Enter a positive integer: "))  
  
y = 0  
  
while (y * y <= n):  
    y = y + 1  
  
print("The smallest y such that y*y > n is:", y)
```

Output

```
enter number: 15  
4
```

Output

```
enter number: 32  
6
```

Output

```
enter number: 49  
8
```

Solving problems via exhaustive search

Try out all possible values until you find the solution

A positive number n is prime if the only numbers that divide it are 1 and n . Write a program that takes as input n , and outputs “Yes” if n is prime.

Write a program that takes as input two positive integers a, b and prints the largest g such that g divides both a and b .

Write a program that takes as input a positive integer n , and prints the smallest y such that $y * y > n$.

Exercise: try out the previously discussed programs for large (15 - 20 digit) inputs. Does your program **always** terminate within 1 second?

Can we do better? Can we have faster programs for these problems?

Doing better by careful exhaustive search

Restrict the search space, perform exhaustive search on the restricted space

Primality testing : can we do better?

A positive number n is prime if the only numbers that divide it are 1 and n . Write a program that takes as input n , and outputs “Yes” if n is prime.

```
n = int(input("enter number: "))  
  
i = 2  
  
prime = True  
  
while(i < n):  
    if (n%i == 0):  
        prime=False  
        break  
  
    i = i+1  
  
if(prime == True):  
    print(n,"is prime")  
else:  
    print(n,"is not prime")
```

Observation: if n is not prime, then it has a factor less than or equal to $\sqrt{n}$

Proof: if n is not prime, then $n = a * b$ for some integers a, b . Both a and b cannot be larger than $\sqrt{n}$.

Primality testing : can we do better?

```
n = int(input("enter number: "))

i = 2

prime = True

while(i<n):

    if (n%i == 0):
        prime=False
        break

    i = i+1

if(prime == True):
    print(n,"is prime")
else:
    print(n,"is not prime")
```

```
n = int(input("enter number: "))

i = 2

prime = True

while(i*i <= n):

    if (n%i == 0):
        prime=False
        break

    i = i+1

if(prime == True):
    print(n,"is prime")
else:
    print(n,"is not prime")
```

Exercise: compare both these programs for large (15 - 20 digit) inputs.
Do both programs **always** terminate within 1 second?

Exercise: Largest divisor of a number

Write a program that takes as input n , and outputs the **largest** divisor of n less than n .

```
n = int(input("enter number: "))  
  
i = n-1  
  
while(i > 0):  
    if (n%i == 0):  
        break  
  
    i = i-1  
  
print("Largest divisor is", i)
```

Output

```
enter number: 15  
Largest divisor is 5
```

Output

```
enter number: 153  
Largest divisor is 51
```

Observation: if n is not prime, and i is the smallest divisor (greater than 1) of n , then n/i is the largest divisor of n (less than n).

Doing better than exhaustive search

Greatest Common Divisor (GCD)

Try out all possible values until you find the solution

Write a program that takes as input two positive integers a, b and prints the largest g such that g divides both a and b .

Our program is effectively trying all numbers in the set $\{1, 2, \dots, \min(a, b)\}$ and finding the largest one that divides both a and b

We have used only one fact about gcd:

$$1 \leq \gcd(a, b) \leq \min(a, b)$$

Observation: Suppose $a = b * q + r$

where $q = a // b, r = a \% b$. Then

$\gcd(a, b)$ divides r .

if $r = a \% b$ and $\gcd(a, b)$ divides r ,

$$\text{then } 1 \leq \gcd(a, b) \leq r$$

Euclid's method for GCD



Euclid, 4th century BC

Euclid's method : gcd(144, 89)

Step	a	b	r = a % b
1	144	89	55
2	89	55	34
3	55	34	21
4	34	21	13
5	21	13	8
6	13	8	5
7	8	5	3
8	5	3	2
9	3	2	1
10	2	1	0

Final output

Euclid's method : gcd(1071, 462)

Step	a	b	r = a % b
1	1071	462	147
2	462	147	21
3	147	21	0

Final output

Euclid's method : implementation in Python

Euclid-method:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

r = a % b

while (r != 0):
    a = b
    b = r
    r = a % b

print("GCD is", b)
```

Notations:

(a_i, b_i, r_i) : values of a, b, r
at end of i^{th} iteration

$$a_0 = a, b_0 = b, r_0 = r$$

- 1. Termination:** Why does this process terminate?
- 2. Correctness:** When it terminates, why does it output the gcd of a and b ?
- 3. Efficiency:** is this faster than exhaustive search?

Euclid's method : termination

Euclid-method:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

r = a % b

while (r != 0):
    a = b
    b = r
    r = a % b

print("GCD is", b)
```

Notations:

(a_i, b_i, r_i) : values of a, b, r
at end of i^{th} iteration

$$a_0 = a, b_0 = b, r_0 = r$$

Observation 1: For all i , $b_i < b_{i-1}$.

Observation 2: For all i , $b_i > 0$

If the inputs are a, b ,
then the while loop
terminates in at most
 b iterations.

Euclid's method : correctness

Euclid-method:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

r = a % b

while (r != 0):
    a = b
    b = r
    r = a % b

print("GCD is", b)
```

Notations:

(a_i, b_i, r_i) : values of a, b, r
at end of i^{th} iteration

$$a_0 = a, b_0 = b, r_0 = r$$

$$\gcd(a, b) = g$$

Observation 3: For all i , $\gcd(a, b)$ divides a_i, b_i, r_i .

Observation 4: $\gcd(a, b)$ divides the output of the above

program

But this is not enough, we want to show that the output of this
program is **equal** to $\gcd(a, b)$

Euclid's method : correctness

Euclid-method:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

r = a % b

while (r != 0):
    a = b
    b = r
    r = a % b

print("GCD is", b)
```

Notations:

(a_i, b_i, r_i) : values of a, b, r
at end of i^{th} iteration

$$a_0 = a, b_0 = b, r_0 = r$$

$$\gcd(a, b) = g$$

Observation 4: $\gcd(a, b)$ divides the output of the above

But this is not enough, we want to show that the output of this program is **equal** to $\gcd(a, b)$

Observation 5: If output of the above program is d ,
then for all i , d divides a_i, b_i, r_i . Therefore d divides a, b .

Using Observation 4 and Observation 5, we conclude output of the program is **equal** to $\gcd(a, b)$

Euclid's method : efficiency

Euclid-method:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

r = a % b

while (r != 0):
    a = b
    b = r
    r = a % b

print("GCD is", b)
```

Notations:

(a_i, b_i, r_i) : values of a, b, r
at end of i^{th} iteration

$$a_0 = a, b_0 = b, r_0 = r$$

$$\gcd(a, b) = g$$

Claim: For all i , $b_{i+2} < b_i/2$.

Therefore, the program terminates in at most $2\log_2(b)$ iterations

Case 1: $r_i \leq b_i/2$

Case 2: $r_i > b_i/2$

Proof: Then $b_{i+2} < b_{i+1} = r_i \leq b_i/2$

Then $r_{i+1} = a_{i+1} \% b_{i+1} = b_i \% r_i < b_i/2$
 $b_{i+2} = r_{i+1} < b_i/2$

Doing better than exhaustive search

Square root of a number : Exhaustive Search

Write a program that takes as input a positive integer n , and prints the smallest y such that $y * y > n$

```
n = int(input("Enter a positive integer: "))  
  
y = 0  
  
while (y * y <= n):  
    y = y + 1  
  
print("The smallest y such that y*y > n is:", y)
```

Output

```
enter number: 15  
4
```

Output

```
enter number: 32  
6
```

Output

```
enter number: 49  
8
```

This program requires $\approx \sqrt{n}$ iterations. Try this program for 25 digit numbers, how long does the program take?

Square root of a number : Binary Search

Write a program that takes as input a positive integer n , and prints the smallest y such that $y * y > n$

In exhaustive search, our initial search space is $\{1, 2, \dots, n\}$.

In each iteration, we reduce the search space by 1 element.

Binary search : we reduce the search space by half in each iteration.

Square root of a number : Binary Search

Square-root-binary-search.py:

```
n = int(input())

left, right = 0, n + 1

while (left + 1 < right):
    mid = (left + right) // 2

    if (mid * mid > n):
        right = mid
    else:
        left = mid

print(right)
```

1. **Termination:** Why does this process terminate?
2. **Correctness:** When it terminates, why does it output the smallest y such that $y * y > n$?

Binary Search: Termination and Correctness

Square-root-binary-search.py:

```
n = int(input())

left, right = 0, n + 1

while (left + 1 < right):
    mid = (left + right) // 2

    if (mid * mid > n):
        right = mid
    else:
        left = mid

print(right)
```

Termination: At end of every iteration, $(\text{right} - \text{left})$ *reduces*.

When the program terminates, $\text{left} + 1 = \text{right}$

Correctness: At end of every iteration,
 $(\text{left} * \text{left}) \leq n < (\text{right} * \text{right})$

Modify the previous program to compute $\lfloor \sqrt{n} \rfloor$

```
n = int(input())

left, right = 0, n + 1

while (left + 1 < right):
    mid = (left + right) // 2

    if (mid * mid > n):
        right = mid
    else:
        left = mid

print(right)
```

Takes as input a positive integer n ,
prints the smallest y such that $y * y > n$

n

```
n = int(input())

left, right = 0, n + 1

while (left + 1 < right):
    mid = (left + right) // 2

    if (mid * mid > n):
        right = mid
    else:
        left = mid

print(left)
```

Takes as input a positive integer n ,
prints the largest y such that $y * y \leq n$